

Slices

Arrays are **extremely strict**, and their **size can never change**. In actual software development, you rarely know exactly how much data you will get. This is why Go created the **Slice**.

1. What is a Slice?

A **Slice** is not an array itself. A Slice is simply a flexible "**window**" that looks (points) at an underlying, hidden Array. Because the **Slice** is just a window, you can stretch it, shrink it, or pass it around without ever having to **move or copy** the heavy underlying data.

2. The Three Properties of a Slice

Under the hood, every Slice is just a tiny data structure made of three things:

1. **Pointer**: The memory address pointing to the exact slot in the underlying array where the slice's "**window**" begins.
2. **Length (len)**: How many elements are currently visible inside the window.
3. **Capacity (cap)**: How much **room is left in the underlying array starting from the pointer**. This is the absolute maximum the window can stretch before it runs out of room.

3. Creating Slices using the Bracket Notation

You can create a slice by looking at an existing array and using the colon.

```
x := [...]int{10, 20, 30, 40, 50, 60}
s1 := x[0:3] // slice of x from index 0 to 2
```

Remember: The slice includes the starting index, but excludes the ending index. **e.g.**, The slice **s1** looks at **index 1 (10)** and **index 2 (20)**. It stops before **index 3**.

- The **Length** of s1 is **2**.
- The **Capacity** of s1 is **6** (because there are 6 total elements from **10** to the end of the array).

Remember: If a slice starts from **index 3** of the array, **then its capacity is 3 because there are 3 elements left in the array**.

4. The Danger of Overlapping Slices

Because Slices are just windows looking at the same underlying array, multiple slices can overlap.

Warning: If you modify data through one slice, it physically changes the underlying array. Any other slice looking at that same spot will instantly see the changed data.

In the example below, there is **x[2] (30)** common in both slices. **If once slice changes, the original underlying array will be changed**, and both slices will now see the **updated value**.

```
x := [...]int{10, 20, 30, 40, 50, 60}
```

```
s1 := x[0:3] // [10, 20, 30]
```

```
s2 := x[2:5] // [30, 40, 50]
```

```
s1[2] = 100
```

```
fmt.Println(s2[0]) // 100
```

5. Slice Literals (The "Normal" Way)

In **99%** of Go code, you won't manually create an array and then slice it. You will just use a **Slice Literal**.

```
slice := []int{10, 20, 30, 40, 50}
```

- Notice that the **square brackets []** are **completely empty!** (No numbers, no ... means a slice).
- **What Go does automatically:** When Go sees empty brackets, it secretly creates a hidden **array** of size 3 in memory, and then instantly creates a **Slice** that points to that array. **The Length and Capacity is same when we use slice literals.**

Expert Takeaway:

If you are building a **dashboard** that displays a list of active users, you will store that list as a Slice of user structs. When you pass that Slice to a function that filters out inactive users, you aren't cloning massive amounts of data in memory; you are just passing around a **tiny 24-byte window (Pointer, Length, and Capacity)** that looks at the original data. This is how Go achieves incredible performance with minimal RAM usage.